DevCollab — Enterprise SaaS Platform System Architecture, API Specification & Technical Interview Guide

Author: Ayush Anand | Stack: React 19, Express 5, MongoDB Atlas, Socket.IO, Docker
Live App: https://devcollab-ten-black.vercel.app | Repo: https://github.com/imayush1304/devcollab

1. Executive Summary & Live Deployment Topology

DevCollab is an enterprise-grade, real-time project management platform inspired by Jira, Linear, and Trello. Engineered for high-concurrency environments, it features zero-trust Role-Based Access Control (Admin, Member, Viewer), Optimistic Concurrency Control (OCC) using Mongoose document versioning (__v) to prevent lost updates, bidirectional low-latency state synchronization with Socket.IO room isolation, and an exhaustive 80-test automated verification suite.

> 📌 **LIVE PRODUCTION DEPLOYMENT & VERIFIED ACCESS**
> Frontend Application (Vercel): https://devcollab-ten-black.vercel.app
> Backend API & WebSockets (Render): https://devcollab-8zpg.onrender.com
> Health & Cluster Diagnostics: https://devcollab-8zpg.onrender.com/api/health
> GitHub Source Repository: https://github.com/imayush1304/devcollab
> Demo Accounts: Admin (ayush@example.com), Member (sarah.dev@example.com), Viewer (alex.viewer@example.com) — Password: Password123!

2. End-to-End System Architecture

The system decouples state mutation from real-time state broadcast by utilizing a dual-transport paradigm: Stateless HTTPS REST for atomic CRUD operations and persistent WSS (WebSockets) for delta broadcasts.

Layer	Technologies	Responsibilities & Engineering Guarantees
Client Presentation	React 19, Vite, @dnd-kit, Custom CSS Glassmorphism	Optimistic drag-and-drop, state cache, rollback handling, audio-visual notification indicators.
Security & Reverse Proxy	Nginx, Helmet, Express Rate Limiter, CORS Firewall	IP rate limiting (5000 dev / 300 prod), origin whitelisting (*.vercel.app), XSS/MIME/Clickjacking protection.
Application Backend	Node.js 22, Express 5, Clean Service/Controller Pattern	Zero-trust RBAC middleware, transactional business logic, OCC version resolution, input validation schemas.
Real-Time Engine	Socket.IO Server, Handshake JWT Authenticator	Room subscription management (workspace:<id>, user:<id>), sub-50ms delta broadcasting, zero cross-tenant leakage.
Data Persistence	MongoDB Atlas Replica Set, Mongoose 8.x ODM	Compound indexed query acceleration, atomic findOneAndUpdate version locking, audit log retention.

3. Database Schema & Data Models

The database leverages MongoDB Atlas with 8 strictly typed schemas and custom compound indexes:

Model	Key Attributes & Types	Index Optimization	Cardinality & Relationships
User	_id, name (str), email (uniq str), password (bcrypt 12), avatar	{ email: 1 } (Unique)	1-to-N Memberships, Tasks assigned, Comments created
Workspace	_id, name, slug (uniq str), description, ownerId (FK)	{ slug: 1 } (Unique)	1-to-N Memberships, Projects, Tasks, and Activities
Membership	_id, userId (FK), workspaceId (FK), role (Admin/Member/Viewer)	{ userId: 1, workspaceId: 1 } (Unique)	Maps User to Workspace with strict RBAC privilege levels
Project	_id, name, key (str), description, workspaceId (FK)	{ workspaceId: 1, key: 1 }	Belongs to Workspace; contains 1-to-N Kanban Tasks
Task	_id, title, status (enum), priority, order (float), __v (int), projectId, workspaceId, assigneeId	{ projectId: 1, status: 1, order: 1 }	OCC versioned document; belongs to Project & Workspace; has N Comments
Comment	_id, taskId (FK), authorId (FK), content (str), createdAt	{ taskId: 1, createdAt: 1 }	Threaded comment on specific task; triggers assignee alerts
Activity	_id, workspaceId (FK), userId (FK), action (enum), entityType, details	{ workspaceId: 1, createdAt: -1 }	Immutable audit timeline log for compliance and board history
Notification	_id, recipientId (FK), actorId (FK), type, title, message, read (bool)	{ recipientId: 1, read: 1, createdAt: -1 }	Real-time user alert; fast unread count aggregation

4. Comprehensive REST API Specification

All protected endpoints enforce JWT authentication via HttpOnly cookies or Bearer Authorization headers.

Method	Endpoint Route	RBAC Access	Payload / Parameters	Success Response
POST	/api/auth/register	Public	{ name, email, password }	201 Created + Token & User Object
POST	/api/auth/login	Public	{ email, password }	200 OK + JWT Cookie & User Profile

POST	/api/auth/logout	Authenticated	None	200 OK + Cleared Auth Cookie
GET	/api/auth/me	Authenticated	None	200 OK + Current User Profile
GET	/api/workspaces	Authenticated	None	200 OK + List of user's active workspaces
POST	/api/workspaces	Authenticated	{ name, description }	201 Created (Creator assigned Admin)
GET	/api/workspaces/:id	Viewer+	params: id	200 OK + Workspace details & member metrics
POST	/api/workspaces/:id/members	Admin	{ email, role }	200 OK + Membership created/invited
DELETE	/api/workspaces/:id/members/:uid	Admin	params: id, uid	200 OK + Member removed
GET	/api/projects/workspaces/:wid	Viewer+	params: wid	200 OK + Projects list with task counts
POST	/api/projects/workspaces/:wid	Member+	{ name, key, description }	201 Created + New Project
GET	/api/tasks/projects/:pid	Viewer+	params: pid, ?status, ?priority	200 OK + Grouped/Sorted Kanban tasks
POST	/api/tasks/projects/:pid	Member+	{ title, description, priority, assigneeId }	201 Created + Broadcast task:created
PATCH	/api/tasks/:id	Member+	{ status, order, expectedVersion }	200 OK (or 409 Conflict if __v mismatch)
DELETE	/api/tasks/:id	Admin/Creator	params: id	200 OK + Broadcast task:deleted
GET	/api/tasks/:id/comments	Viewer+	params: id	200 OK + Comment thread with authors
POST	/api/tasks/:id/comments	Member+	{ content }	201 Created + Broadcast notification
GET	/api/notifications	Authenticated	query: ?unreadOnly=true	200 OK + Notifications &

				unread count
PATCH	/api/notifications/:id/read	Recipient	params: id	200 OK + Marked read
PATCH	/api/notifications/read-all	Recipient	None	200 OK + All user alerts marked read
GET	/api/health	Public	None	200 OK + DB connection & uptime status

5. Core Algorithms & Critical Code Implementations

5.1 Optimistic Concurrency Control (OCC) with Mongoose __v

To prevent lost updates when two team members reorder or drag tasks concurrently, we implement OCC. The update query atomically checks if the document version matches the client's cached version. If a concurrent modification has occurred, the version check fails, returning null, which triggers an HTTP 409 Conflict.

```
// Atomic OCC Update Query
// server/src/services/task.service.js
const updateTask = async (taskId, updates, expectedVersion, userId, workspaceId) => {
  const query = { _id: taskId, workspaceId };
  if (expectedVersion !== undefined) {
    query.__v = expectedVersion; // Lock condition
  }

  const updatedTask = await Task.findOneAndUpdate(
    query,
    { $set: updates, $inc: { __v: 1 } }, // Atomically bump version
    { new: true, runValidators: true }
  );

  if (!updatedTask) {
    // Either task was deleted or version was bumped by another client
    const current = await Task.findById(taskId);
    if (current && expectedVersion !== undefined && current.__v !== expectedVersion) {
      throw new ConflictError('Task was modified by another user. Reloading latest state...');
    }
    throw new NotFoundError('Task not found');
  }
  return updatedTask;
};
```

5.2 Fractional / Floating-Point Kanban Card Ordering

Traditional SQL reordering updates all N rows when moving an item (O(N) write cost). DevCollab uses fractional positioning: new_order = (prev_order + next_order) / 2.0. Moving a card between 1000 and 2000 results in 1500, requiring only a single document write (O(1)). If the difference drops below 0.0001, an asynchronous re-normalization re-spreads items evenly.

```
// O(1) Fractional Card Positioning
// client/src/pages/KanbanBoard.jsx
const calculateNewOrder = (items, destinationIndex) => {
  if (items.length === 0) return 1000;
  if (destinationIndex === 0) return items[0].order / 2;
  if (destinationIndex >= items.length) return items[items.length - 1].order + 1000;
  const prev = items[destinationIndex - 1].order;
  const next = items[destinationIndex].order;
  return (prev + next) / 2;
};
```

5.3 WebSocket Room Isolation & Handshake Authentication

Socket connections verify JWT credentials during the initial WebSocket HTTP handshake. Upon verification, the socket joins exclusively authorized rooms: `workspace:<id>` and `user:<id>`. State changes in Workspace A are never broadcast to Workspace B.

```
// Isolated Socket Handshake & Tenant Broadcast
// server/src/sockets/socketHandler.js
io.use(async (socket, next) => {
  try {
    const token = socket.handshake.auth?.token || socket.handshake.headers?.cookie?.jwt;
    if (!token) return next(new Error('Authentication required'));
    const decoded = verifyToken(token);
    socket.user = decoded;
    next();
  } catch (err) {
    next(new Error('Invalid token'));
  }
});

// Scoped room broadcast
io.to(`workspace:${workspaceId}`).emit('task:updated', { task: updatedTask, actorId });
```

5.4 Zero-Trust RBAC Middleware (Admin, Member, Viewer)

Every API route verifies workspace membership and checks the role hierarchy before controller execution:

```
// Role Hierarchy Verification Middleware
// server/src/middleware/workspaceAuth.js
const ROLE_WEIGHTS = { Admin: 3, Member: 2, Viewer: 1 };

export const requireRole = (minRole) => {
  return async (req, res, next) => {
    const { workspaceId } = req.params;
    const membership = await Membership.findOne({ userId: req.user.id, workspaceId });
    if (!membership) throw new ForbiddenError('Not a member of this workspace');
    if (ROLE_WEIGHTS[membership.role] < ROLE_WEIGHTS[minRole]) {
      throw new ForbiddenError(`Action requires ${minRole} privilege level`);
    }
    req.membership = membership;
    next();
  };
};
```

6. Frontend Engineering & UX Innovations

The frontend is built with React 19 and custom Vanilla CSS design tokens (glassmorphism, subtle micro-animations):

• Dual Authentication Resilience: Authenticates using HttpOnly SameSite=None cookies with an Axios request interceptor providing Bearer fallback from localStorage for non-browser clients.

• Optimistic UI with Rollback: Kanban moves immediately update React state. If the server throws 409 Conflict, the board reverts to the previous snapshot and displays an interactive toast notification.

• Pointer & Touch Sensor Calibration: Configured @dnd-kit sensors with 8px drag activation threshold to distinguish intentional drags from simple clicks.

• Real-Time Notification Bell: Displays live unread badge, smooth dropdown animation, direct card-click focus, and mark-all-read capability.

7. Automated Testing Architecture

Test Suite File	Tests Count	Scope & Critical Scenarios Covered
health.test.js	3 tests	Server uptime, diagnostic database status, ping latency.
auth.test.js	12 tests	Registration validation, Bcrypt 12 salt rounds, JWT expiry, invalid credentials.
workspace.test.js	10 tests	Multi-tenancy isolation, unique slug generation, member invitations, owner safeguards.
project_task.test.js	10 tests	Project hierarchy, task lifecycle (TODO->IN_PROGRESS->DONE), priority filtering.
comment_activity.test.js	9 tests	Threaded comments, author profile population, activity timeline audit entries.
socket.test.js	6 tests	WebSocket upgrade handshake, room joining security, cross-tenant isolation.
notification.test.js	9 tests	In-app alerts on task assignment, unread count caching, mark-all-read mutations.
edge_cases.test.js	21 tests	Simultaneous write collisions, duplicate slugs, invalid ObjectIDs, RBAC escalation attempts.
TOTAL SUITE	80 / 80 PASS	100% test pass rate across unit, integration, concurrency, and security suites.

8. Product-Based Company Interview Guide (All Scopes)

This section prepares you for Tier-1 engineering interviews (Google, Amazon, Uber, Atlassian, Microsoft, Meta) by covering every architectural dimension of DevCollab across 6 distinct interview scopes.

[Scope 1: Concurrency & Distributed Systems]

Q1: How do you prevent lost updates when two users move the same Kanban task simultaneously?

Answer: We implement Optimistic Concurrency Control (OCC) using Mongoose document versioning (__v). When an update is initiated, the client sends the task's expected version. The backend query executes:
Task.findOneAndUpdate({ _id: taskId, __v: expectedVersion }, { $set: updates, $inc: { __v: 1 } })
If user B updated the card first, the version is already incremented. The database query finds 0 matching documents and returns null. The backend throws an HTTP 409 Conflict. The client catches this, displays an informative toast, and rolls back the optimistic UI move while re-fetching the canonical document from the server. This avoids heavy distributed database locking (Pessimistic 2PL) while guaranteeing zero data loss.

[Scope 1: Concurrency & Distributed Systems]

Q2: Why did you choose Optimistic Locking over Pessimistic Locking?

Answer: In web-scale collaborative SaaS applications, human think-time is measured in seconds/minutes, while database transactions take milliseconds. If User A opens a modal and we hold a pessimistic lock (e.g., SELECT ... FOR UPDATE or mutex), all other users are blocked from even viewing or commenting on that task until User A releases the lock. Optimistic locking assumes collisions are infrequent (~1%), enabling high read and write throughput without database deadlocks. If collisions do occur, OCC fails fast with 409 Conflict.

[Scope 2: WebSockets & Real-Time Scalability]

Q3: How do you scale Socket.IO across multiple server instances in production?

Answer: In our current setup, Socket.IO manages rooms in-memory on a single Node.js instance. To scale horizontally across multiple instances behind an AWS ALB / Nginx load balancer:
1. Redis Pub/Sub Adapter (@socket.io/redis-adapter): When an instance emits 'task:updated' to room 'workspace:123', the event is published to a shared Redis channel. All other server instances receive the pub/sub message and broadcast it to their locally connected clients.
2. Sticky Sessions: Since WebSockets start as an HTTP polling handshake before upgrading, the load balancer uses ip_hash or a cookie to route polling packets to the same instance until WebSocket upgrade completes.

[Scope 2: WebSockets & Real-Time Scalability]

Q4: How did you ensure zero cross-tenant data leakage over WebSockets?

Answer: Socket.IO events are never broadcast globally via io.emit(). We enforce strict Room Isolation:
1. Handshake JWT authentication ensures every socket has a verified user identity (socket.user).
2. When a user navigates to a workspace, the client emits 'workspace:join'. The server queries Membership to verify that socket.user.id is indeed an active member of that workspace before calling socket.join(`workspace:${workspaceId}`).
3. All mutations (task created, moved, deleted, comments) emit strictly to io.to(`workspace:${workspaceId}`).emit(). Clients in other workspaces are physically unaware of and cannot listen to those packets.

[Scope 3: Database & Performance Optimization]

Q5: How did you optimize MongoDB queries for high-volume Kanban boards?

Answer: We established compound indexes mapped directly to query equality, sort, and range access patterns (ESR Rule):
1. Tasks: Index { projectId: 1, status: 1, order: 1 } ensures that loading columns and sorting cards is satisfied entirely from index memory (B-tree index scan) with 0 in-memory sorting overhead.
2. Activity Timeline: Index { workspaceId: 1, createdAt: -1 } allows instantaneous pagination for audit history without scanning un-related workspace records.
3. Notifications: Index { recipientId: 1, read: 1, createdAt: -1 } allows sub-5ms lookup for the unread notification bell indicator.

[Scope 3: Database & Performance Optimization]

Q6: Why use fractional floating-point ordering over integer indexing for Kanban cards?

Answer: If tasks use integer positions (1, 2, 3, 4, ...), dragging task 100 to position 2 requires shifting 98 documents in the database: UPDATE tasks SET order = order + 1 WHERE order >= 2. That is an O(N) write operation with high

write contention and write amplification. With fractional ordering, moving an item between item A (order: 1000) and item B (order: 2000) calculates new_order = (1000 + 2000)/2 = 1500. This requires updating exactly 1 document (O(1) write cost). If floating-point precision delta drops below 0.0001, a background worker re-normalizes the column.

[Scope 4: Security & Authentication]

Q7: Explain your dual authentication strategy (HttpOnly Cookie vs Bearer Header).

Answer: In modern web applications, storing JWTs exclusively in localStorage leaves the application vulnerable to token theft via Cross-Site Scripting (XSS). We set HttpOnly, Secure, SameSite=None session cookies so browser scripts cannot read the token. However, in cross-domain environments, mobile apps, or headless API clients, third-party cookie blocking can occasionally occur. Therefore, our Axios interceptor also attaches 'Authorization: Bearer <token>' as a secondary layer. The server's auth middleware checks: const token = req.cookies.jwt || req.headers.authorization?.split(' ')[1]. This achieves maximum enterprise security and cross-platform resilience.

[Scope 4: Security & Authentication]

Q8: How does your Role-Based Access Control (RBAC) work across workspaces?

Answer: We employ a zero-trust multi-tenant RBAC model with three hierarchical roles: Admin (level 3), Member (level 2), and Viewer (level 1).
Privileges are scoped per workspace via the Membership collection, not stored on the global User document. This allows Ayush to be an Admin in 'Engineering Workspace' while being a read-only Viewer in 'Marketing Workspace'. The requireRole(minRole) middleware verifies membership and evaluates whether ROLE_WEIGHTS[membership.role] >= ROLE_WEIGHTS[minRole]. Critical administrative actions (member removal, workspace deletion) strictly enforce Admin privileges.

[Scope 5: Frontend Architecture & UX]

Q9: What is the difference between Optimistic UI and Pessimistic UI in your Kanban board?

Answer: Pessimistic UI waits for the network response before updating the view. On a 150ms 4G connection, dragging a task card would feel sluggish and freeze. DevCollab uses Optimistic UI: when the drag ends, React state immediately updates the card's column and position locally. Concurrently, an API request is fired in the background. If the request succeeds, state commits silently. If it fails (network error or 409 Conflict), the client catches the error, reverts the card to its original position snapshot, and alerts the user. This delivers the snappy 60fps UX of native desktop apps.

[Scope 5: Frontend Architecture & UX]

Q10: Why did you choose @dnd-kit over older libraries like react-beautiful-dnd?

Answer: react-beautiful-dnd is no longer actively maintained by Atlassian and does not support modern React 18/19 concurrent rendering features. @dnd-kit is modular, headless, lightweight, and built natively with React Hooks. It provides first-class PointerSensor and TouchSensor APIs, collision detection algorithms (closestCorners, rectIntersection), and accessibility (ARIA keyboard navigation) out of the box.

[Scope 6: Behavioral, Edge Cases & Architecture Defense]

Q11: What was the most challenging technical bug you encountered in DevCollab, and how did you resolve it?

Answer: The most challenging issue was handling cross-site cookie blocking between Vercel and Render in production. Initially, cookies had sameSite: 'lax', which browsers silently reject when making cross-origin requests from https://devcollab.vercel.app to https://devcollab.onrender.com. We resolved this by: (1) Setting sameSite: 'none' and secure: true in production, (2) adding app.set('trust proxy', 1) in Express so Express recognizes HTTPS behind Render's reverse proxy, and (3) configuring the dual-fallback Authorization Bearer header in Axios interceptors, ensuring seamless authentication across all network topologies.

[Scope 6: Behavioral, Edge Cases & Architecture Defense]

Q12: If given 3 more months on this codebase, what would you build next?

Answer: Three high-value architectural additions:
1. Redis Caching & PubSub Adapter: Integrate Redis for sub-millisecond workspace metadata caching and multi-instance WebSocket horizontal scaling.
2. CRDTs / Operational Transformation: For collaborative rich-text task descriptions (like Notion/Google Docs).

3. Background Asynchronous Workers (BullMQ + Redis): Offload email notifications, daily digest reports, and file export generation from the main Node.js event loop.